Calculatrice.java

```java
public class Calculatrice {

    public int division(int a, int b) {
        return a / b;
    }
}
```

Lab 02 —

Code Analysis Using PMD

MODULE	Quality Assurance and Software Testing
TOPIC	Software Fault Identification, Defect Density, and Static Code Analysis
PROGRAMMING LANGUAGE	Java
TOOL	PMD

PREPARED BY

Mohamed Yacine Saadia

PROMOTION Master 2 Software Engineering

Lab Objectives

By the end of this lab, students should be able to:

- Identify potential faults and code-quality problems in a Java program.
- Classify identified faults according to the taxonomy studied in the lecture.
- Calculate and interpret defect density.
- Use PMD to perform static code analysis.
- Interpret a static analysis report and understand its limitations.

Assessment 1 — Manual Code Analysis

Given Code

Consider the following Java class, `Calculatrice`:

```java
public class Calculatrice {

    public int division(int a, int b) {
        return a / b;
    }

    public int modulo(int a, int b) {
        return a % b;
    }

    public double moyenne(int[] valeurs) {
        if (valeurs == null) return 0;
        if (valeurs.length == 0) return 0;

        double somme = 0;
        for (int i = 0; i < valeurs.length; i++) {
            somme += valeurs[i];
        }
        return somme / valeurs.length;
    }
}
```

Questions

1. Identify at least five potential faults or code-quality problems in the given class.
2. For each identified problem, explain why it could be problematic.
3. According to the fault taxonomy presented on slide 33 of the lecture, classify each identified problem.
4. Which problems could cause a runtime failure? Which ones primarily affect readability, maintainability, or reliability?
5. Can you confirm that a fault exists simply by looking at the source code, or do you sometimes need additional information about the requirements?

Correction — Assessment 1

No.	Identified problem	Classification
1	Division by zero is not explicitly handled in `division()`.	Design or boundary-condition issue
2	Modulo by zero is not explicitly handled in `modulo()`.	Design or boundary-condition issue
3	The parameters `a` and `b` have unclear names.	Coding convention / readability issue
4	Public methods have no explanatory documentation.	Documentation issue
5	Returning `0` for a null array may hide an invalid input.	Design / error-handling issue
6	Returning `0` for an empty array may be ambiguous because the average is mathematically undefined.	Design / boundary-condition issue
7	The code uses an indexed loop where an enhanced `for` loop could improve readability.	Coding practice / maintainability issue



Important clarification: These are potential problems, not necessarily seven confirmed functional faults. For example, Java already throws an `ArithmeticException` when integer division or modulo uses a zero divisor. Whether this behavior is acceptable depends on the specified requirements. Similarly, returning `0` for a null array may be intentional, although it can make errors harder to detect.

Assessment 2 — Lines of Code and Defect Density

Suppose that a software tester has identified **8 confirmed defects** in the `Calculatrice` class.

Questions

1. Count the lines of code (LOC) in the `Calculatrice` class.
2. For this exercise, assume that the class contains **20 LOC**.
3. Calculate the number of thousands of lines of code (KLOC).

$$\text{KLOC} = \frac{\text{LOC}}{1000}$$

4. Calculate the defect density using the following formula:

$$\text{Defect Density} = \frac{\text{Number of Defects}}{\text{KLOC}}$$

5. Is the resulting defect density considered good? Justify your answer.

Correction — Assessment 2

Step 1 — Determine LOC

The class contains 17 non-empty source lines in the displayed version. However, for this exercise, we use the specified assumption of **20 LOC**.

Step 2 — Convert LOC to KLOC

$$\text{KLOC} = \frac{20}{1000} = 0.02$$

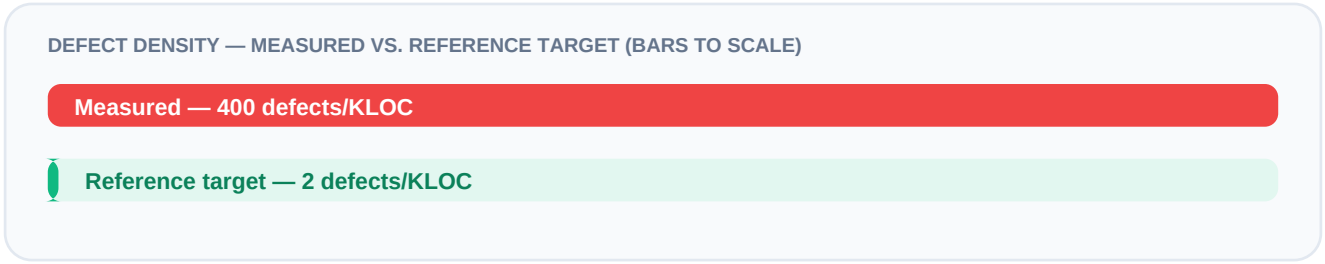
Step 3 — Calculate defect density

$$\text{Defect Density} = \frac{8}{0.02}$$

400 defects/KLOC

Step 4 — Interpret the result

The measured density is high: eight confirmed defects have been identified in only 20 lines of code.



If the course uses **2 defects/KLOC** as a reference target, the measured density is far above that target. The code should therefore be investigated and reviewed carefully.

> ⚠ However, there is no universal defect-density threshold that applies to every software project. The interpretation depends on the project's context, measurement method, severity of defects, and applicable quality criteria.

Assessment 3 — Automated Static Code Analysis Using PMD

Part A — Install PMD

1. Download the PMD distribution from the official repository or release page.
2. Extract the downloaded archive into a directory of your choice.
3. Locate the `bin` directory inside the extracted PMD folder.
4. Save the provided Java class as `Calculatrice.java`.
5. Open a terminal or command prompt and navigate to the PMD `bin` directory.

Official resources:

PMD repository: https://github.com/pmd/pmd

PMD documentation: https://docs.pmd-code.org/

Note: On Windows, you may use a directory such as `C:\PMD`. On macOS, you can use a directory under your home folder. It is not necessary to extract PMD into the root of the drive.

Part B — Run the Analysis

Run PMD using the Java quickstart ruleset.



Windows

```
pmd.bat check -d C:\Calculatrice.java -R rulesets/java/quickstart.xml -f text
```

macOS

From the PMD installation directory, run:

```
./bin/pmd check -d /path/to/Calculatrice.java -R rulesets/java/quickstart.xml -f text
```

Replace `/path/to/Calculatrice.java` with the actual path to the source file.

Understanding the command

Option	Meaning
`check`	Runs the source-code analysis.
`-d`	Specifies the source file or directory to analyze.
`-R`	Specifies the ruleset used for the analysis.
`-f text`	Displays the report in plain text in the terminal.

Part C — Generate an HTML Report

Run the analysis again, this time producing an HTML report.

macOS

```
./bin/pmd check -d /path/to/Calculatrice.java \
  -R rulesets/java/quickstart.xml \
  -f html -r report.html
```

Windows

```
pmd.bat check -d C:\Calculatrice.java -R rulesets/java/quickstart.xml -f html -r report.html
```

The `-f html` option selects HTML output, while `-r report.html` specifies the report file.

On macOS, open the report with:

```
open report.html
```

On Windows, you can open `report.html` in your preferred web browser.

Use the syntax and ruleset path appropriate to the PMD version you installed.

Part D — Analyze and Interpret the Results

After generating the report, answer the following questions:

1. How many violations did PMD report?
2. What types of violations were detected?
3. Which source-code lines are associated with the reported violations?
4. Does PMD detect all the potential faults identified in Assessment 1?
5. Why might a program contain a fault even when PMD reports no violation?
6. Can static analysis replace software testing? Explain your answer.

Expected Conclusion

PMD is a static code-analysis tool: it examines source code without executing the program and identifies violations of selected rules.

It can help developers identify certain coding-practice problems, maintainability issues, and potential defects. However, it does not automatically identify every functional error.

For example, a division-by-zero problem may not be reported by the selected ruleset. The expression `a / b` is syntactically valid Java, but it can fail at runtime when `b` is zero.

Therefore, reliable software quality assurance requires complementary techniques, including:

1. **Manual code review.**
2. **Static code analysis.**
3. **Unit testing and boundary-value testing.**
4. **Dynamic testing and validation against requirements.**

Final question for discussion: If PMD reports zero violations, can we conclude that the software is free of defects? Why or why not?